SIEM Brute-Force Detection & Log Analysis Lab

Splunk Enterprise SIEM Home Lab — Credential Attack Simulation & Detection Engineering

Pavan kumar | Author

1. Project Overview

This project involved building a small Security Information and Event Management (SIEM) home lab using Splunk Enterprise, onboarding real Windows log sources into it, simulating a credential brute-force attack, and engineering a working detection query for it. The objective was not simply to run an attack tool and capture a screenshot, but to understand the underlying log data well enough to separate genuine attack telemetry from normal background authentication noise, and to document the reasoning behind every detection decision.

The lab covers three areas: (1) SIEM infrastructure setup and log source onboarding, (2) attack simulation and iterative detection query development for brute-force / failed-logon activity (MITRE ATT&CK T1110), and (3) field-level log analysis used to validate the detection and eliminate false positives.

2. Lab Architecture

Component	Role
Windows Host (physical machine, hostname BATMAN)	Victim / monitored endpoint. Runs Sysmon for process telemetry and the Splunk Universal Forwarder to ship Windows Security and Sysmon event logs to the SIEM.
Splunk Enterprise (Ubuntu Server 22.04, VirtualBox VM)	Central SIEM. Receives forwarded logs on TCP 9997, indexes them, and runs all search/detection queries (SPL).
Kali Linux (VirtualBox VM)	Attacker platform, used for the initial network-based brute-force attempt against the Windows host.

3. Log Source Onboarding

Two Splunk indexes were created to separate log types: wineventlog for the Windows Security Event Log, and sysmon for Sysmon's Operational log. A receiving port (9997) was opened on the Splunk indexer, and the Splunk Universal Forwarder was installed on the Windows host and pointed at the SIEM's IP address on that port.

The Windows Security Event Log was configured to forward all events, and Sysmon (configured with the SwiftOnSecurity baseline configuration) was set to forward Event ID 1 (process creation) and related telemetry. Ingestion was verified directly in Splunk before any attack activity was generated, confirming both log sources were live and searchable.

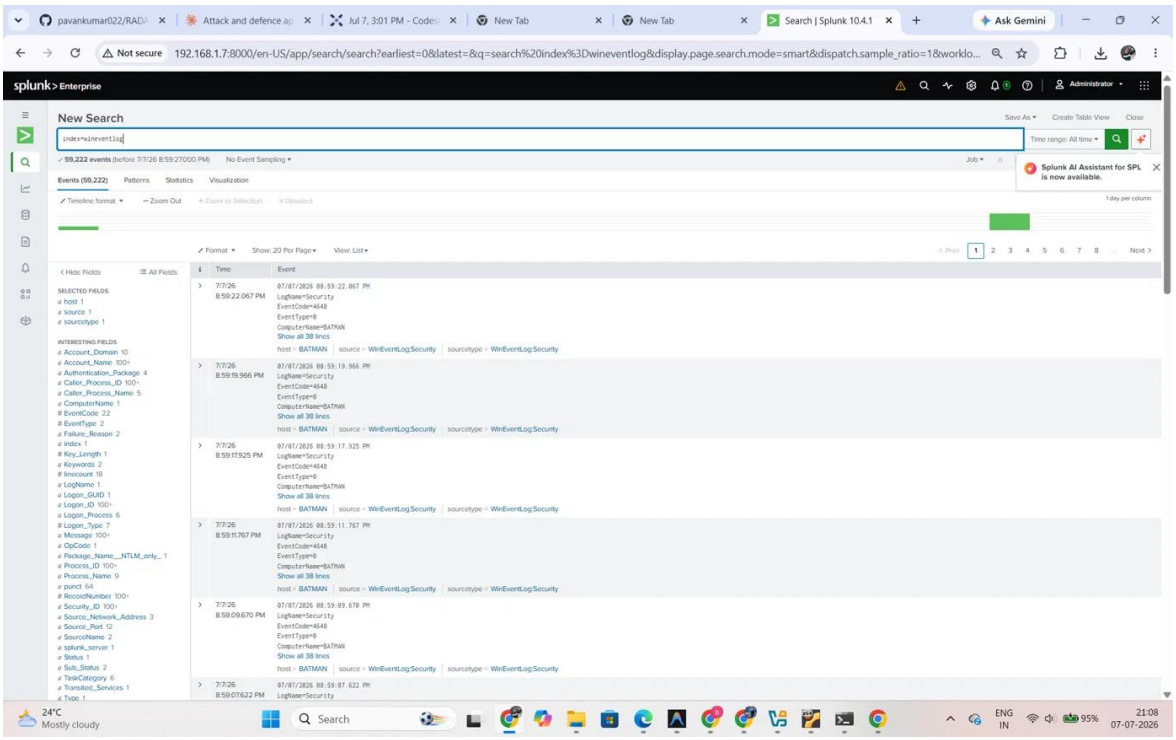

Figure 1. Windows Security Event Log ingestion verified in Splunk — index=wineventlog, 59,222 events received from host BATMAN via the Universal Forwarder.

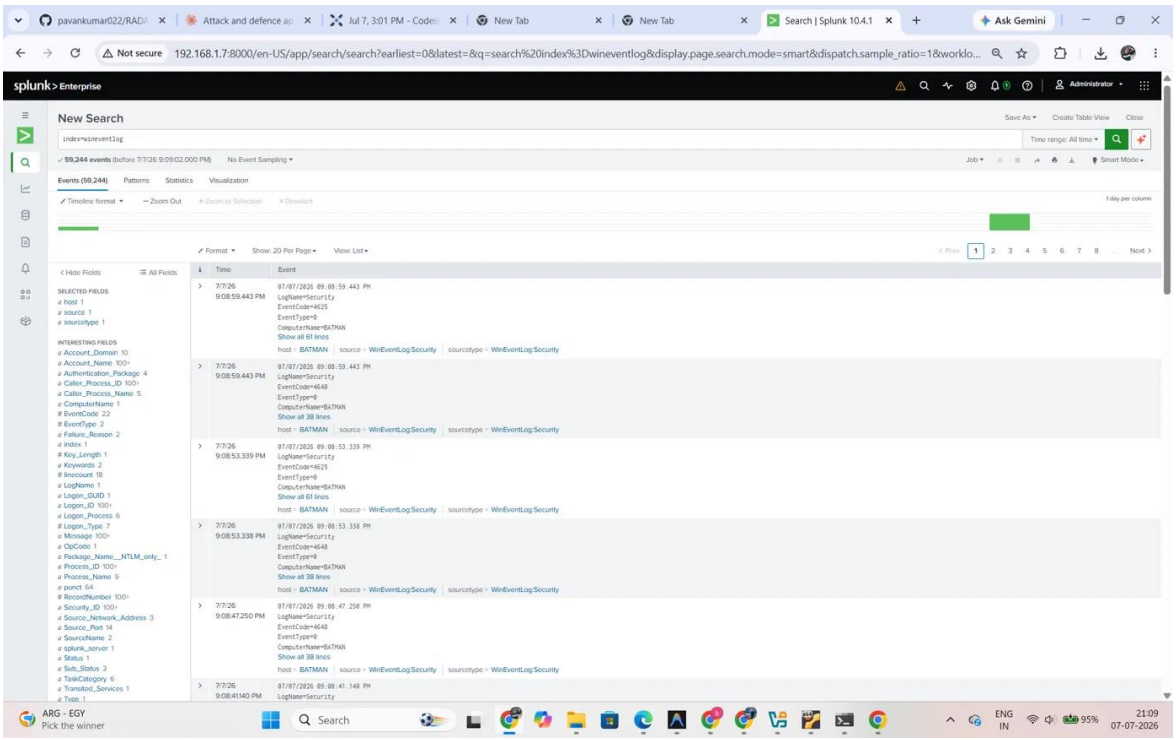

Figure 2. Continued verification of Windows Security Event Log ingestion, showing EventCode 4625 and 4648 events already present prior to the attack simulation.

A configuration issue was identified and corrected during Sysmon onboarding: the forwarder's inputs.conf initially had renderXml = true, which caused Sysmon events to arrive as raw, unparsed XML blobs instead of individually searchable fields (Image, CommandLine, User, ParentImage, etc.). This was corrected to renderXml = false so that Splunk's Sysmon-aware sourcetype could auto-extract fields, which any Sysmon-based detection query depends on.

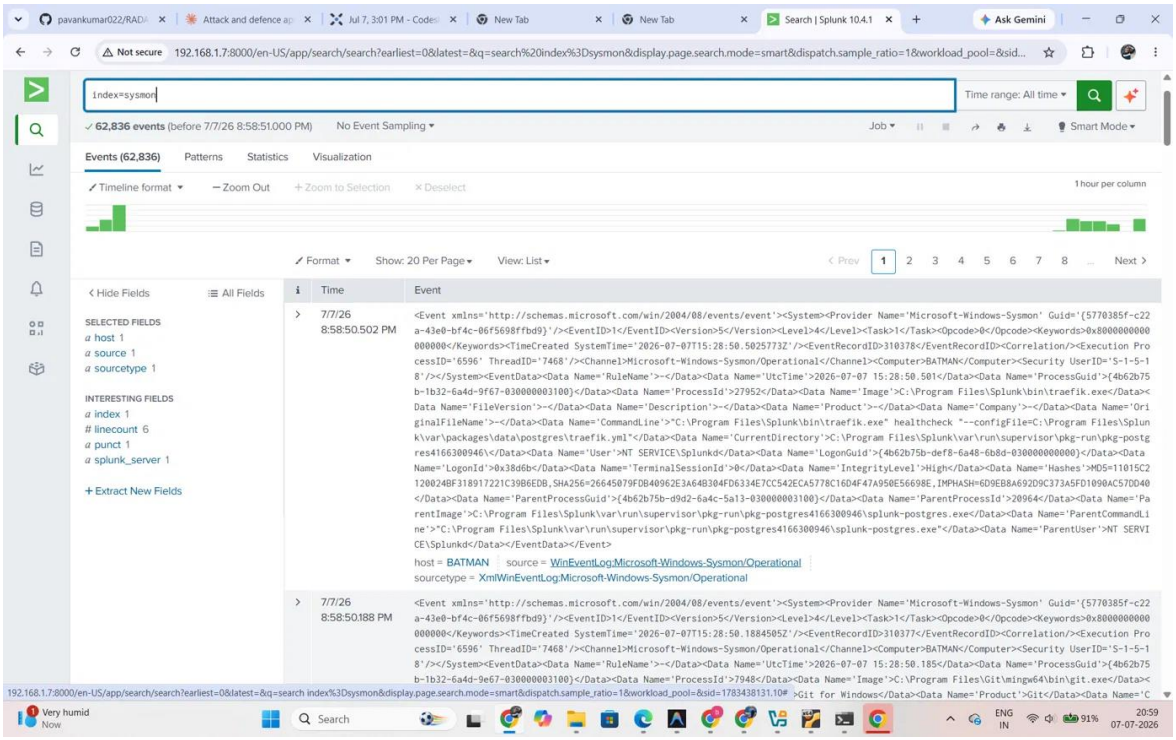

Figure 3. Sysmon Operational log ingestion verified — index=sysmon, 62,836 events received, confirming Sysmon telemetry was reaching the SIEM.

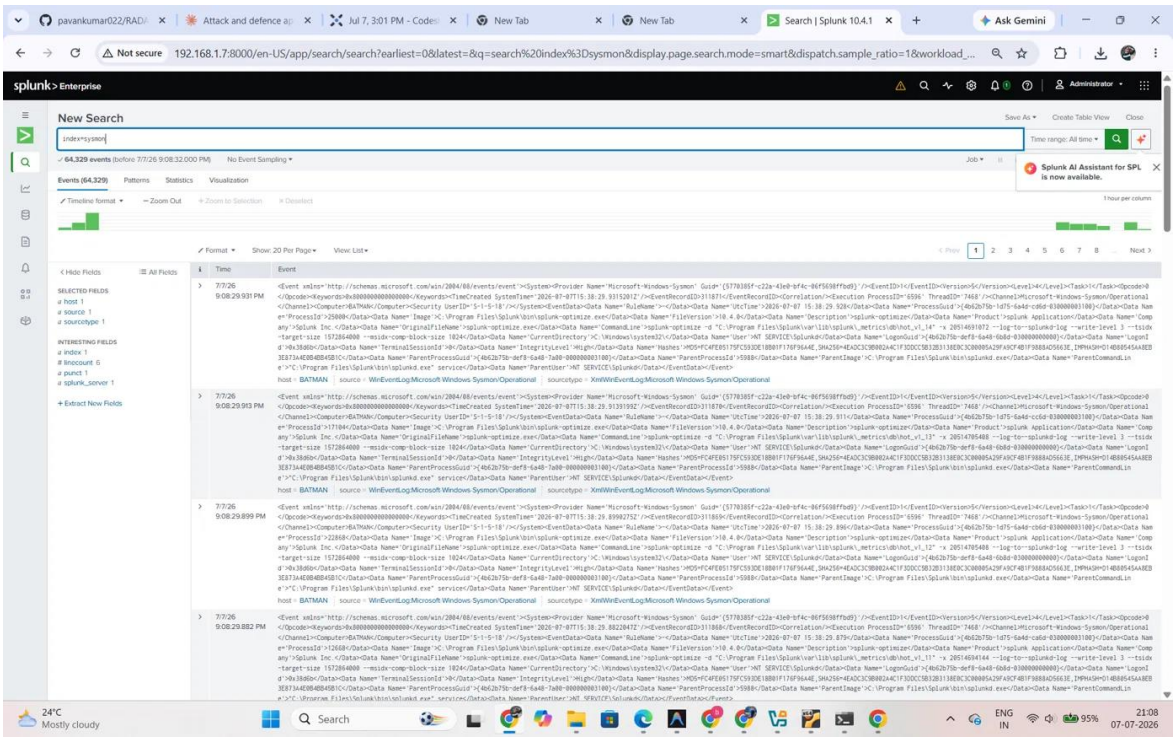

Figure 4. Sysmon ingestion continuing to build (64,329 events), providing process-creation visibility on the Windows host for future detection work.

4. Attack Simulation & Detection Engineering — Brute-Force Login (T1110)

4.1 Initial approach and a documented obstacle

The original plan was to run a dictionary brute-force attack from the Kali VM against the Windows host using Hydra (targeting RDP/SSH). This was attempted first. It was not successful in this environment: Windows Defender and network-stack behavior on the target interfered with the remote attack traffic before enough failed-logon events could be generated for a clean detection test.

Rather than treat this as a dead end, the attack was re-scoped to produce the same evidentiary log data — repeated Windows failed-logon events (Event ID 4625) — through a locally-run authentication loop on the Windows host itself, using a scripted loop of failed SMB share authentications with a fixed username and rotating incorrect passwords. This generates a genuine 4625 event for every failed attempt through the same Windows authentication stack that a remote brute-force tool would trigger, which keeps the detection logic and log fields realistic even though the traffic did not cross the network from Kali.

This limitation and the reasoning behind the pivot are recorded here deliberately: it reflects an actual constraint encountered in the environment and the troubleshooting decision made in response, rather than presenting the attack as something it was not.

4.2 Detection query

The detection query targets repeated failed logons per account within the Windows Security log:

```
index=wineventlog EventCode=4625
| stats count by Account_Name, Source_Network_Address
| where count > 10
```

A threshold of 10 was used rather than a lower number because a handful of genuine mistyped-password attempts by a real user rarely exceeds 3-4 failures in a session, so 10 gives margin against false positives while still catching automated/rapid attempts quickly.

4.3 Results

Running the query against the full log history initially surfaced two account groups exceeding the threshold, not one — this is discussed in detail in Section 5, since correctly interpreting it required examining the underlying fields rather than trusting the aggregate count alone.

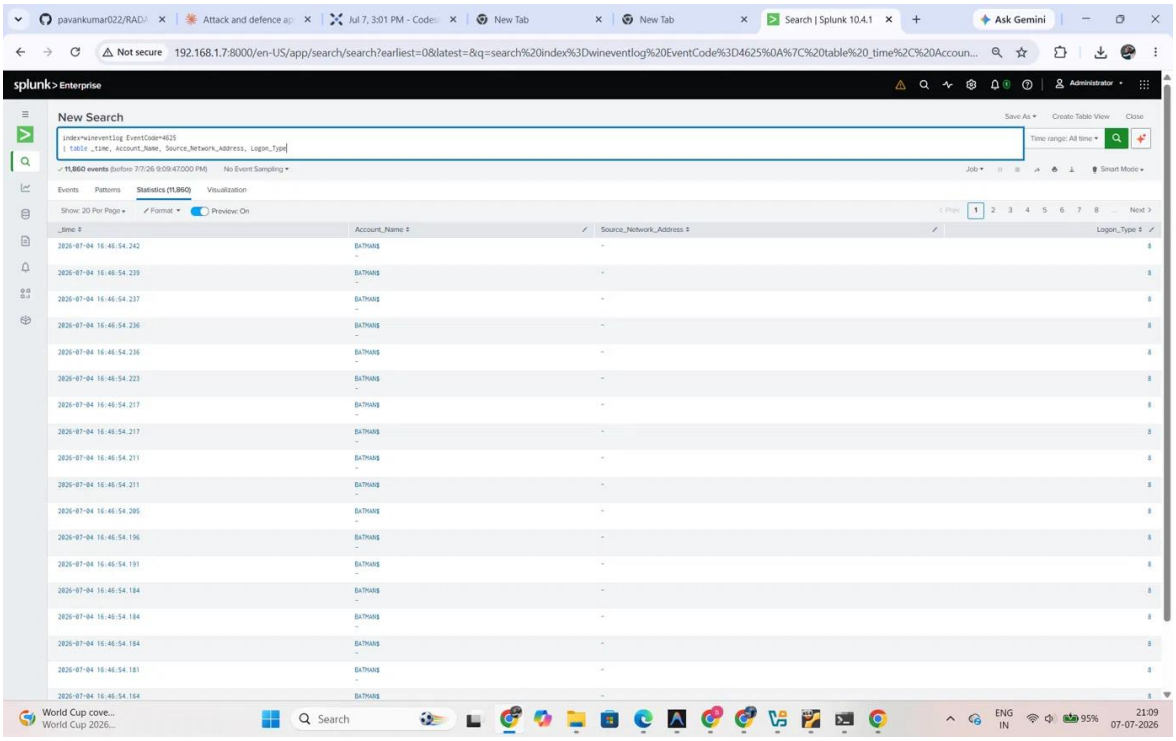

Figure 5. Unfiltered results of the failed-logon query, tabulating _time, Account_Name, Source_Network_Address, and Logon_Type. A high volume of events tied to the BATMAN$ machine account is visible alongside the simulated test activity.

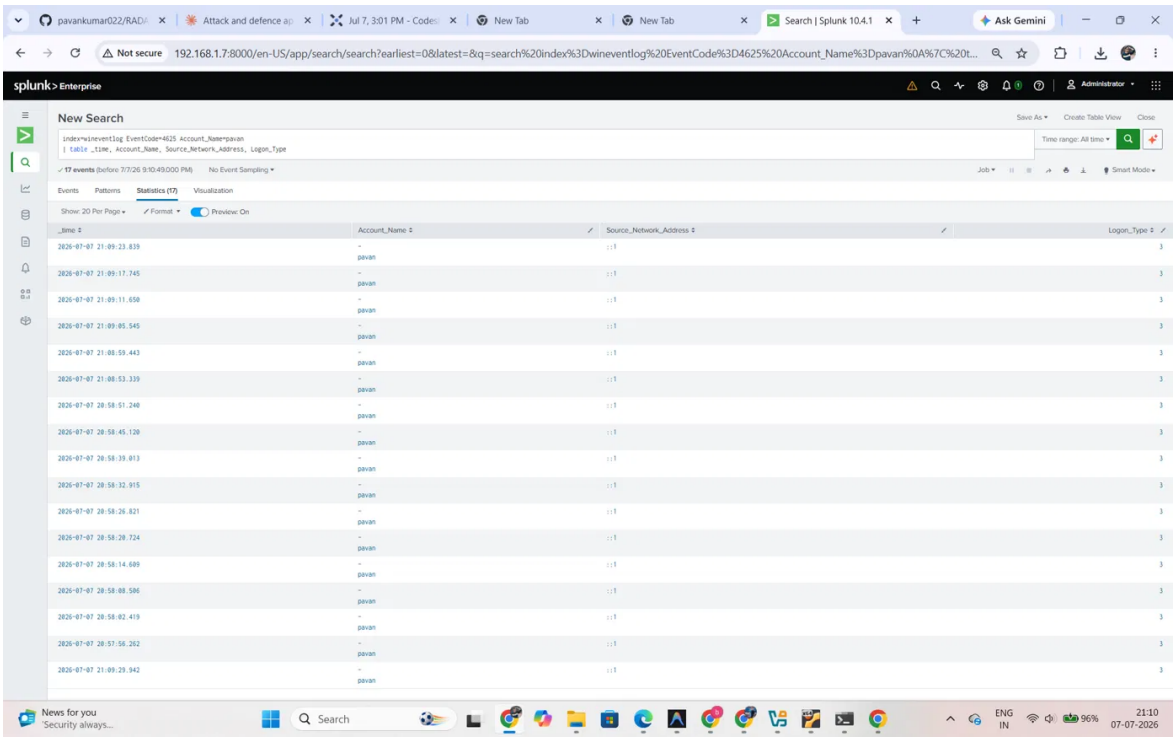

Figure 6. Query filtered to Account_Name=pavan, isolating the 17 failed-logon events generated by the local attack simulation, all with Logon_Type 3 and Source_Network_Address ::1 (loopback).

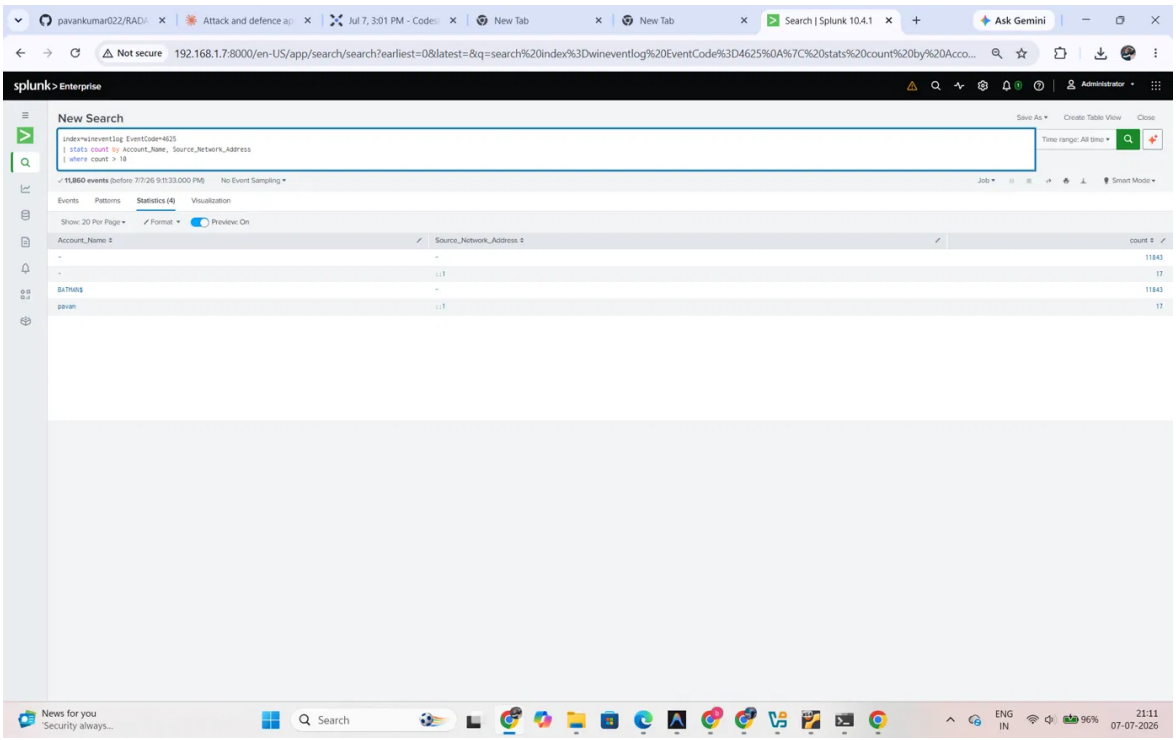

Figure 7. Aggregated stats by Account_Name and Source_Network_Address confirming two distinct groups: BATMAN$ (11,843 events, background noise) and pavan (17 events, the simulated attack).

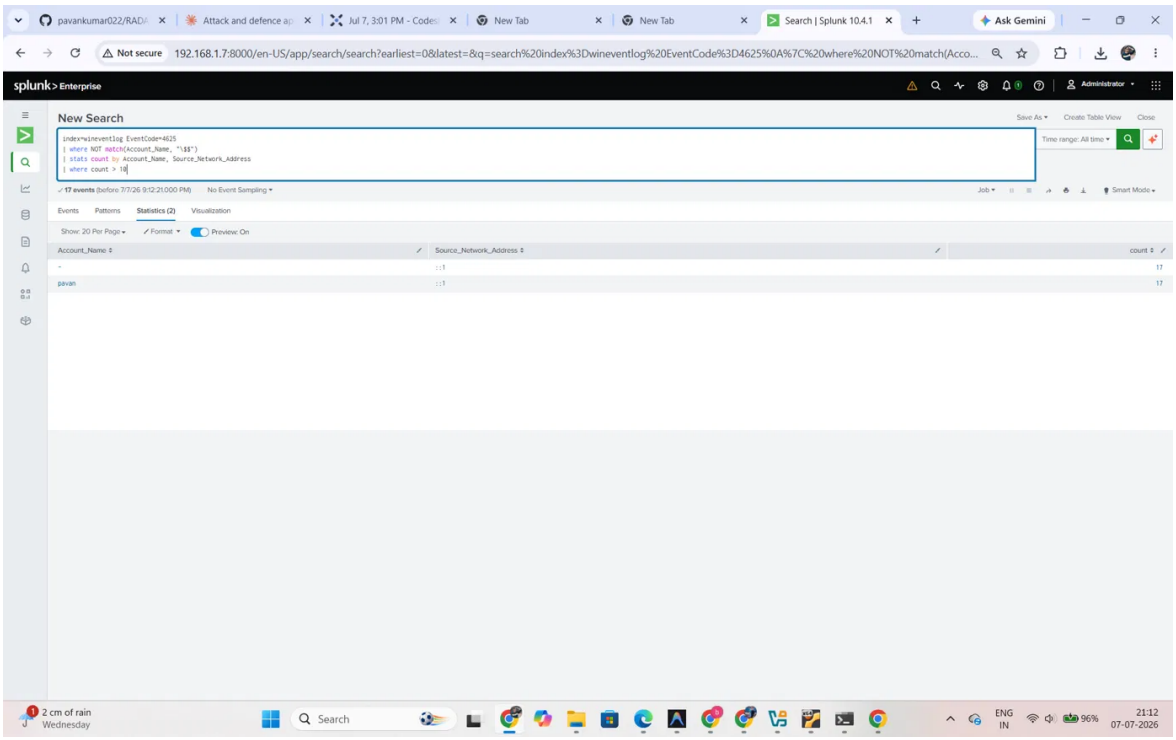

Figure 8. Final, noise-excluded detection query — Account_Name filtered to exclude machine accounts (trailing $) — correctly isolating the pavan account's 17 failed-logon attempts as the detected brute-force activity.

The final, refined detection query excludes Windows machine/service accounts (which are always suffixed with $) so that background authentication noise does not mask or dilute a genuine account-based brute-force alert:

```
index=wineventlog EventCode=4625
| where NOT match(Account_Name, "\\$")
| stats count by Account_Name, Source_Network_Address
| where count > 10
```

5. Log Analysis — Understanding the Data

Getting a detection query to return a number is the easy part; understanding whether that number means what it appears to mean is the part that matters. This section documents the field-level analysis used to validate the detection above.

5.1 Event codes: 4625 vs. 4648

Event ID 4625 (An account failed to log on) is the direct evidence of a failed authentication attempt and is the correct event for brute-force detection. Event ID 4648 (Logon using explicit credentials) also appeared heavily in the ingested log and was initially a source of confusion — it records a different action (a process explicitly supplying alternate credentials, common in normal Windows service behavior) and is not itself a failed-logon indicator, so it was excluded from the detection logic.

5.2 Account_Name: user accounts vs. machine accounts

The unfiltered query (Figure 5) initially returned two account groups above the count-10 threshold: pavan (the real test account) and BATMAN$ (the local computer/machine account, denoted by the trailing $ that Windows automatically appends to computer and service accounts). Treating both as equally significant would have been a mistake. Cross-checking the timestamps showed the BATMAN$ failures were dated several days prior to the test and occurred continuously in the background — consistent with a local service or scheduled process repeatedly failing to authenticate, unrelated to the attack simulation. This is a realistic and common pattern in production SIEM data: a naive threshold-only rule will false-positive on routine service-account noise unless it is explicitly filtered out.

5.3 Logon_Type and Source_Network_Address

Two supporting fields were used to confirm the test data was correctly attributed: Logon_Type 3 (Network) is consistent with the SMB-based authentication method used in the simulation, whereas the BATMAN$ noise carried Logon_Type 8 (NetworkCleartext), a different authentication path. Source_Network_Address showed ::1 (the IPv6 loopback address) for the simulated attack events, which correctly reflects that the attempts originated from the same host rather than an external attacker IP — an expected and honestly-documented artifact of the local simulation method described in Section 4.1, and a field that would need to show a real external address in a genuine remote brute-force scenario.

5.4 Why this matters

Filtering machine accounts, distinguishing 4625 from 4648, and validating Logon_Type and Source_Network_Address together turned a single aggregate count into a defensible detection: 17 failed logons against one real user account, from one consistent source, in a tight time window — clearly distinguishable from 11,843 unrelated background failures from a machine account. This kind of noise-versus-signal reasoning is the difference between a query that merely runs and a query that would hold up in an actual SOC alert triage.

6. MITRE ATT&CK Mapping

Activity	Technique ID	Tactic
Repeated failed-logon / brute-force credential attempts	T1110 (Brute Force)	Credential Access

7. Key Findings and Lessons Learned

- A remote, cross-VM brute-force attempt can be blocked by endpoint defenses and network-stack behavior even in a controlled lab; the underlying detection logic can still be validated by reproducing the same authentication event locally, provided the pivot and its limitations are documented honestly rather than presented as the original attack.
- Raw event counts are not sufficient for a reliable detection — Windows machine accounts (trailing $) generate high volumes of routine failed authentications that must be filtered out, or a threshold-based rule will alert on noise instead of, or in addition to, real activity.
- Logon_Type and Source_Network_Address are necessary corroborating fields, not optional extras — they were what actually confirmed which events belonged to the simulated attack and which were unrelated background activity.

- Correct Sysmon field extraction depends on forwarder-level configuration (renderXml); a log source can appear to be "working" while still being unusable for detection if it is left in raw XML form.
- A count-based threshold (count > 10) is a reasonable first-pass detection but has a known blind spot: an attacker who throttles attempts below the threshold, or spreads them across a longer time window, would evade it. A time-windowed version of the same query (e.g., using span-based stats) would be a stronger next iteration.

LinkedIn : pavankumar022

Github: pavankumar022

E-mail: pavankumar797524@gmail.com